

Angewandte Mathematik

Vorlesung 2 von 10

Lean und Typentheorie: Terme, Typen, Regeln

Skript: Abschnitte 3.1–3.5 und 3.7

Ablauf heute

0:00	Rückblick; ab jetzt ausführlich; Lean und Mathlib	Skript 3, Einleitung
0:10	Teil 1: Lean lesen und schreiben	Begleitcode, Teil 0
0:22	Terme, Typen, Universen	Skript 3.1
0:34	Abhängige und induktive Typen; Gleichheit und Axiome	Skript 3.1, 3.2, 3.4
0:58	Urteile und Regeln	Skript 3.5
1:15	*Pause*	
1:30	Teil 2: Paare, Summen, Gleichheit; Herleitungen	Skript 3.5
1:48	Curry–Howard: warum ein Term ein Beweis ist; `Prop`	Skript 3.3, 3.1
2:18	Rechnen, Prüfen, Vertrauen; Term- und Taktikmodus	Skript 3.5, 3.7
2:27	Zusammenfassung und Ausblick	
2:30	Ende	

Rückblick: die Einführung

- **Beweisassistent.** Die Aussage ist ein Typ, der Beweis ein Term. „Ist der Beweis richtig?“ wird zu „hat dieser Ausdruck diesen Typ?“ – eine Rechnung, die ein kleiner *Kern* ausführt.
- **Russell.** Uneingeschränkte Komprehension ist widersprüchlich: $R := \{\, x \mid x \notin x \,\}$ liefert $R \in R \iff R \notin R$.
- **Cantor.** Es gibt keine Menge aller Mengen.
- **Formale Systeme.** Zeichen, Formeln, Axiome, Schlussregeln – Prämissen oben, Konklusion unten. Hilbert: viele Axiome, eine Regel. Gentzen: Einführung und Beseitigung, Annahmen werden entlassen.
- **λ-Kalkül.** Variable, Abstraktion, Anwendung; gerechnet wird mit $(\lambda x.\, M)\, N \rightsquigarrow M[N/x]$. Konfluent – aber $(\lambda x.\, x\, x)\, (\lambda x.\, x\, x)$ bricht nie ab.

Ab jetzt ausführlich

Termin 1: die Landkarte

Motivation, naive Mengenlehre, formale Systeme, λ-Kalkül, Syntax und Semantik, Gödel, ZFC und NBG, Kategorien.

Im Überblick: Ideen, Anschauung, Zusammenhänge – Formales nur exemplarisch.

Termine 2–10: gründlich

- Definitionen und Sätze mit allen Voraussetzungen
- Beweise und Rechnungen Schritt für Schritt
- Lean-Code dazu, vom Compiler geprüft
- Live-Demos im Editor

Heute: Lean lesen und schreiben, Leans Grundlage, die Typentheorie, und wie der Kern prüft.

Eine dritte Antwort

Auf das Komprehensionsproblem kennt man zwei klassische Antworten. Es gibt eine dritte, und auf ihr beruht Mathlib:

- ZFC schränkt ein, *was man bilden darf*.
- NBG schränkt ein, *was Element sein darf*.
- Die Typentheorie schränkt ein, *worüber eine Formel überhaupt reden kann*.

Der Unterschied zu ZFC ist der *Zeitpunkt*:

ZFC	**Typentheorie**
Ersetzt Komprehension durch *Aussonderung*: Man darf nur aus einer Menge aussondern, die es schon gibt.	Verlangt, dass jede Komprehension einen *Typ* angibt, über den sie läuft.
Ein Axiomenschema löst das Problem. Unzulässige Bildungen sind gültige, aber unbeweisbare Aussagen.	Die Syntax löst das Problem. Unzulässige Bildungen sind gar keine Ausdrücke.

Lean und Mathlib

- **Lean 4 mit Mathlib** (getestet mit Toolchain und Mathlib in Version `v4.26.0`).
- Mathlib ist *eine* Bibliothek statt vieler Aufsätze: über eine Million Zeilen, mehrere hundert Mitwirkende, täglich erweitert.
- Ihre Grundlage ist nicht ZFC, sondern eine abhängige Typentheorie, der *Calculus of Inductive Constructions*.
- Jeder Codeblock auf den Folien stammt unverändert aus einer Datei in `code/`, die der Compiler akzeptiert hat; welche, steht rechts über dem Code. Pseudo-Code gibt es nicht.

Ergänzend. Eine kurze Einführung in Lean steht in Teil 0.1–0.4 von `code/Typentheorie.lean`. Ausführlicher, mit vielen Beispielen zur Taktiksprache: `LeanMath/basics/lean_typetheory_curry_howard_intro.lean`.

Vorlesung 2

Lean lesen und schreiben

Lean lesen: drei Befehle

Lean `code/Typentheorie.lean`

```
#check 2 + 3              -- ein Ausdruck vom Typ ℕ
#eval 2 + 3               -- 5
#check (2 : ℤ) - 5        -- Typangabe: dieselben Ziffern als ganze Zahlen
#eval (2 : ℤ) - 5         -- -3; in ℕ ergaebe 2 - 5 die 0
#print Nat                -- ℕ ist ein induktiver Typ mit zero und succ
```

- `#check` nennt den Typ eines Ausdrucks, `#eval` rechnet ihn aus, `#print` zeigt, wie ein Name definiert ist. Die Antwort steht im Infofenster, sobald der Cursor auf der Zeile steht.
- `--` beginnt einen Kommentar bis zum Zeilenende, `/- ... -/` einen längeren. `/-- ... -/` vor einer Definition ist ihr Docstring.
- `(2 : ℤ)` legt den Typ fest. In $\mathbb{Z}$ ergibt $2 - 5$ die -3; in $\mathbb{N}$ gibt es keine negativen Zahlen, dort ergibt $2 - 5$ die 0.

Live in Lean `code/Typentheorie.lean` – Teil 0.1: den Cursor auf die Befehle setzen und das Infofenster lesen

Definitionen und Funktionen

Lean code/Typentheorie.lean

```
def verdopple (n : ℕ) : ℕ := 2 * n
#check (verdopple)                -- verdopple : ℕ → ℕ
#eval verdopple 21                -- 42
#check fun x : ℕ => x + 1         -- eine Funktion ohne Namen
#eval (fun x : ℕ => x + 1) 41     -- 42
def addiere (m n : ℕ) : ℕ := m + n
#check (addiere)                  -- addiere : ℕ → ℕ → ℕ
#check addiere 2                  -- ℕ → ℕ: wartet auf das zweite Argument
#eval addiere 2 40                -- 42
```

- `def` Name, Argumente, Ergebnistyp, dann `:=` und der Rumpf. Angewandt wird ohne Klammern: `verdopple 21`. `#check (verdopple)` zeigt den Typ, ohne Klammern zeigt `#check` die Signatur.
- `fun x => x + 1` ist die Abbildung $x \mapsto x + 1$, im λ-Kalkül $\lambda x.\, x + 1$. Das Infofenster schreibt $\mapsto$.
- $\mathbb{N} \to \mathbb{N} \to \mathbb{N}$ heißt $\mathbb{N} \to (\mathbb{N} \to \mathbb{N})$: `addiere 2` ist wieder eine Funktion, die auf das zweite Argument wartet.

Fallunterscheidung, Zwischenergebnisse, Paare

Lean code/Typentheorie.lean

```
def betrag (z : ℤ) : ℤ := if z < 0 then -z else z
def quadratsumme (a b : ℕ) : ℕ :=
  let qa := a * a
  let qb := b * b
  qa + qb
#eval betrag (-7)                  -- 7
#eval quadratsumme 3 4             -- 25
def tausche {α β : Type} (p : α × β) : β × α := (p.2, p.1)
#eval tausche (1, "eins")          -- ("eins", 1): Lean findet α und β selbst
#check @tausche ℕ String           -- α und β ausdruecklich angegeben
```

- `if ... then ... else` unterscheidet Fälle, `let` benennt Zwischenergebnisse, in Beweisen ebenso `have`.
- $\alpha \times \beta$ ist der Typ der Paare, `p.1` und `p.2` sind die Komponenten.
- Argumente in geschweiften Klammern sind *implizit*: Lean ergänzt sie selbst, hier $\mathbb{N}$ für α und `String` für β. Mit `@` gibt man sie ausdrücklich an.

Aussagen und Beweise

Lean code/Typentheorie.lean

```
#check 2 < 3                        -- eine Aussage, vom Typ Prop
#check ∀ n : ℕ, n < n + 1           -- auch mit Quantor

theorem stets_wahr : True := True.intro
theorem stets_wahr_trivial : True := trivial

example (p q : Prop) (hp : p) (hq : q) : p ∧ q := ⟨hp, hq⟩
example (p q : Prop) (hq : q) : p → q := fun _ => hq
```

- Eine Aussage ist ein Ausdruck vom Typ `Prop`, etwa `2 < 3`.
- Ein Beweis von `P` ist ein Term vom Typ `P`. `theorem` gibt ihm einen Namen, `example` prüft ihn nur; `trivial` beweist `True`.
- Ein Beweis von `p ∧ q` ist ein Paar `⟨hp, hq⟩`, ein Beweis von `p → q` eine Funktion. Warum das genau zur Logik passt, erklärt Curry–Howard.

Beweise mit Taktiken

Lean code/Typentheorie.lean

```
example (p q : Prop) (hp : p) (hq : q) : p ∧ q := by
  constructor
  · exact hp
  · exact hq

example (p q : Prop) (hq : q) : p → q := by
  intro _
  exact hq
```

- Nach by entsteht der Beweis Schritt für Schritt. Das Infofenster zeigt das offene Ziel hinter ⊢, darüber die Annahmen.
- `constructor` zerlegt `p ∧ q` in zwei Ziele; ein Punkt · bearbeitet das nächste. `exact` schließt ein Ziel mit einem Term, `intro` nimmt eine Annahme in den Kontext, `apply h` führt mit `h : p → q` das Ziel `q` auf `p` zurück.
- Weitere Taktiken: `induction` für induktive Typen, `rfl`, `simp` und `decide` für Gleichungen, `norm_num` und `omega` für Zahlen.

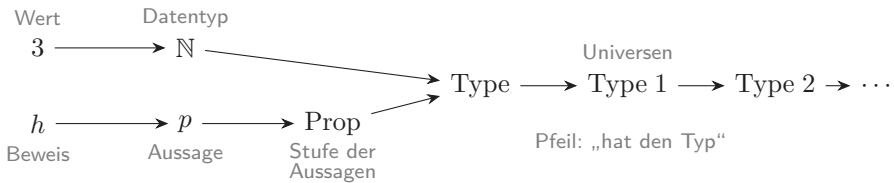
Live in Lean `code/Typentheorie.lean` – Teil 0.4: das Ziel nach jeder Taktik

Vorlesung 2

Terme, Typen, Universen

Terme und Typen

- Es gibt *Terme*, und jeder Term hat genau einen *Typ*: $t : T$.
- Anders als in der Mengenlehre ist $t : T$ keine Aussage, über die man streiten kann, sondern eine Eigenschaft der Syntax: Entweder der Typcheck geht durch, oder der Ausdruck ist keiner.
- Typen sind selbst Terme, haben also Typen – gestaffelt in *Universen*: `Nat : Type`, `Type : Type 1`, `Type 1 : Type 2`, ...; `Type` ohne Index ist `Type 0`.
- Eine Stufe fällt aus der Reihe: `Prop`, die Stufe der *Aussagen*.



Terme und Typen in Lean

Lean code/Typentheorie.lean

```
/- Jeder Ausdruck hat einen Typ; es gibt kein untypisiertes "Universum". -/
#check (3 : ℕ)
#check ℕ
#check Type
#check Prop

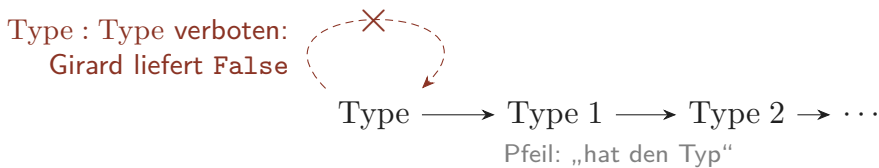
/- Typen leben selbst wieder in Typen, gestaffelt in Universen. -/
#check Type       -- Type : Type 1
#check Type 1     -- Type 1 : Type 2

/- Eine Stufe ist besonders: ‘Prop‘, die Stufe der Aussagen.
   Aussagen sind Typen wie andere auch -- man kann ueber sie quantifizieren. -/
#check Prop
example (p q : Prop) : Prop := p → q
```

- `#check` zeigt den Typ, die Kommentare nennen die Antwort.
- Aus `p q : Prop` wird die Aussage `p → q` – Aussagen sind Typen wie andere auch und zugleich selbst Terme (vom Typ `Prop`).

Die Staffelung ist nötig

- Gegenstück zum Satz von Cantor: Wie es keine Menge aller Mengen gibt, so ist auch die Stufe aller Typen nie ein Objekt ihrer eigenen Stufe.
- **Girards Paradoxon** (1972): Ließe man zu, dass die Stufe aller Typen ein Objekt ihrer selbst ist, wäre das System widersprüchlich – man könnte `False` herleiten.
- Die Universenhierarchie ist also nicht Vorsicht, sondern Notwendigkeit.



Eine feste Stufe reicht nicht

Lean `code/Typentheorie.lean`

```
def idNull {α : Type} (a : α) : α := a
example : ℕ := idNull 3          -- α ist ℕ, und ℕ : Type
-- example : Type := idNull ℕ    -- Fehler: α waere Type, aber Type : Type 1
def idEins {α : Type 1} (a : α) : α := a
example : Type := idEins ℕ       -- α ist Type, und Type : Type 1
-- example : ℕ := idEins 3       -- Fehler: α waere ℕ, aber ℕ : Type
```

- `example : T := t` prüft `t : T`. `idNull` verlangt α `: Type`. Bei `idNull 3` setzt Lean $\mathbb{N}$ für α ein, und $\mathbb{N}$ `: Type` passt. Bei `idNull` $\mathbb{N}$ wäre α der Typ von $\mathbb{N}$, also `Type`, und verlangt wäre `Type : Type` – es gilt aber `Type : Type 1`.
- `idEins` ist dieselbe Definition eine Stufe höher. Sie nimmt $\mathbb{N}$, aber nicht mehr 3: Jeder Term hat genau einen Typ, und der von $\mathbb{N}$ ist `Type`, nicht `Type 1`.
- Jede Definition und jeder Satz über beliebige Typen bräuchte so eine Kopie pro Stufe, und die Stufen enden nie. Ein Universum aller Typen enthielte sich selbst, wie `Type : Type`; das wäre widersprüchlich (Girard).

Live in Lean `code/Typentheorie.lean` – die auskommentierten Zeilen einkommentieren; in der Meldung ist `?m.1` ein noch unbekannter Term, „of sort X“ heißt „liegt in X“

Universenpolymorphie

Was α für die Typen ist, ist u für die Stufen

Lean code/Typentheorie.lean

```
universe u v
def meineId {α : Type u} (a : α) : α := a
#check @meineId.{u}
#check @meineId.{0}
#check @meineId.{1}
example : ℕ := meineId 3
example : Type := meineId ℕ
```

Bei jeder Verwendung setzt Lean selbst eine Stufe für u ein, durch Abgleich:

Aufruf	$a : \alpha$	$\alpha : \mathrm{Type}\ u$	u
`meineId 3`	$3 : \mathbb{N}$	$\mathbb{N} : \mathrm{Type}\ 0$	0
`meineId ℕ`	$\mathbb{N} : \mathrm{Type}$	$\mathrm{Type} : \mathrm{Type}\ 1$	1

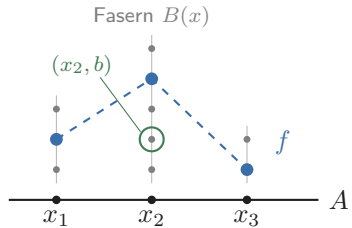
- `universe u v` führt *Universenvariablen* ein: Platzhalter für den Index in `Type 0`, `Type 1`, `Type 2`, ...; `Type u` ist das Universum mit dem Index `u`. Dieser Index ist kein Term, auch keine Zahl aus $\mathbb{N}$: `u` hat keinen Typ, `#check u` ist ein Fehler.
- Damit ist `meineId` *universenpolymorph*: Mit 0 für `u` erhält man die Definition von `idNull`, mit 1 die von `idEins`. Ein Universum aller Typen braucht es dafür nicht.
- `.{...}` gibt die Stufe ausdrücklich an, so wie `@` die impliziten Argumente angibt. Mit `@` zeigt `#check` auch α: {α `: Type u`}, {α `: Type`} bzw. {α `: Type 1`}.

Vorlesung 2

Abhängige und induktive Typen

Abhängige Typen

- Ein Typ darf von einem *Wert* abhängen – der Hauptvorteil.
- `Fin n`, die Zahlen $< n$, hängt von n ab. Eine Funktion, die zu n ein Element von `Fin n` liefert, hat den *abhängigen Funktionstyp* `(n : ℕ) → Fin n`.
- Mengentheoretisch ist ein abhängiger Funktionstyp ein Produkt $\prod_{x \in A} B(x)$, ein abhängiges Paar eine disjunkte Vereinigung $\sum_{x \in A} B(x)$. Daher Π-*Typ* und Σ-*Typ*.



f wählt in *jeder* Faser ein Element (Π);
ein Paar (x_2, b) ist *ein* Punkt (Σ).

Lean `code/Typentheorie.lean`

```
/-- Ein **abhaengiger** Funktionstyp: der Ergebnistyp haengt vom Argument ab. -/
def diagonale : (n : ℕ) → (Fin n → Fin n) := fun _ i => i
```

Länge im Typ

Lean code/Typentheorie.lean

```
inductive Vektor (α : Type u) : ℕ → Type u where
  | nil : Vektor α 0
  | cons : α → {n : ℕ} → Vektor α n → Vektor α (n + 1)
def Vektor.head {α : Type u} {n : ℕ} : Vektor α (n + 1) → α
  | .cons a _ => a
```

- `ℕ → Type u`: zu jeder Länge n ein eigener Typ `Vektor α n`. `nil` hat Länge 0, `cons` macht aus Länge n die Länge $n + 1$.
- `Vektor.head` verlangt Länge $n + 1$: nur für nichtleere Vektoren definiert – und das steht im *Typ*.
- Wendet man `Vektor.head` auf den leeren Vektor `Vektor.nil` an, gibt es keinen Laufzeitfehler, sondern einen *Typfehler*: `Vektor.nil` hat den Typ `Vektor α 0`, verlangt ist `Vektor α (n + 1)`. Der Aufruf kommt gar nicht zustande.

Induktive Typen

Fast alle Typen in Lean sind *induktiv*: Aus den Konstruktoren baut Lean den *Rekursor*.

Lean code/Typentheorie.lean

```
inductive MeinNat where
  | zero : MeinNat
  | succ : MeinNat → MeinNat
  deriving Repr

def MeinNat.add : MeinNat → MeinNat → MeinNat
  | n, .zero => n
  | n, .succ m => .succ (MeinNat.add n m)

def zro : MeinNat := MeinNat.zero
#check MeinNat.succ zro

#eval MeinNat.succ zro
/- Der automatisch erzeugte Rekursor ist zugleich das Induktionsprinzip. -/
#check @MeinNat.rec
```

- `MeinNat` hat zwei Konstruktoren, `zero` und `succ`, sonst nichts – gebaut wie $\mathbb{N}$.
- `MeinNat.add` rekursiert über das *zweite* Argument: $n + 0 \rightsquigarrow n$ und $n + (m + 1) \rightsquigarrow (n + m) + 1$.
- `#check @MeinNat.rec` zeigt den Rekursor – Rekursion und Induktion in einem.

Vorlesung 2

Gleichheit

Zwei Gleichheitsbegriffe

Definitorische Gleichheit

Zwei Terme sind gleich, weil der Kern sie durch Rechnen ineinander überführt. Der Beweis ist `rfl` und kostet nichts.

Beispiel: `2 + 2 = 4`.

Propositionale Gleichheit

Zwei Terme sind gleich, weil es einen Beweis gibt.

Beispiel: `0 + n = n` für variables `n`. Hier muss man induzieren.

Der Grund: `Nat.add` rekursiert wie `MeinNat.add` über das *zweite* Argument. Deshalb rechnet $n + 0$ von selbst, $0 + n$ mit variablem n dagegen nicht.

Gleichheit in Lean

Lean code/Typentheorie.lean

```
/- Definitorische Gleichheit: der Kern rechnet selbst. -/
example : 2 + 2 = 4 := rfl
example (n : ℕ) : n + 0 = n := rfl

/- Propositionale Gleichheit: braucht einen Beweis per Induktion. -/
example (n : ℕ) : 0 + n = n := Nat.zero_add n

/- Funktionale und propositionale Extensionalitaet sind Saetze bzw. Axiome. -/
#check @funext
#check @propext
#check @Quot.sound
```

- Oben: `rfl` – der Kern rechnet $2 + 2$ und $n + 0$ selbst aus.
- Mitte: kein `rfl`, sondern ein Beweisterm – der Satz `Nat.zero_add`, angewandt auf `n`.
- Unten: funktionale und propositionale Extensionalität. Laut Kommentar ist die eine ein Satz (`funext`), die andere ein Axiom (`propext`); dazu `Quot.sound`.

Rechnen oder Induzieren

Lean code/Typentheorie.lean

```
theorem MeinNat.add_zero (n : MeinNat) : n.add .zero = n := rfl

theorem MeinNat.zero_add (n : MeinNat) : MeinNat.zero.add n = n := by
  induction n with
  | zero => rfl
  | succ k ih => simp [MeinNat.add, ih]
```

- `MeinNat.add_zero`: Das zweite Argument ist der Konstruktor `zero`, die erste Gleichung greift, $n + 0 \rightsquigarrow n$: `rfl`.
- `MeinNat.zero_add`: Das zweite Argument ist die *Variable* n – keine Gleichung greift. Also Induktion:
  - Fall `zero`: $0 + 0 \rightsquigarrow 0$, `rfl`.
  - Fall `succ k`: $0 + (k + 1) \rightsquigarrow (0 + k) + 1$; mit `ih`, also $0 + k = k$, erledigt das `simp`.
- Derselbe Rekursor erzeugt Programme und Beweise – kein Zufall: Curry–Howard, nach der Pause.

Live in Lean `code/Typentheorie.lean` – `#check @MeinNat.rec`; `zero_add`: `rfl`, dann `induction`

Vorlesung 2

Die Axiome

Drei Axiome

Über die Regeln hinaus nimmt Lean genau drei Aussagen ohne Beweis an:

`propext`	`(a ↔ b) → a = b` äquivalente Aussagen sind gleich
`Quot.sound`	`r a b → Quot.mk r a = Quot.mk r b` im Quotienten werden verwandte Elemente gleich
`Classical.choice`	`Nonempty` $\alpha \to \alpha$ aus „es gibt ein Element“ wird ein Element

- `propext` erlaubt, eine Aussage überall durch eine äquivalente zu ersetzen.
- `Quot.sound` baut Quotienten: Restklassen G/H, Multimengen als Listen bis auf die Reihenfolge, $\mathbb{R}$ als Cauchy-Folgen modulo Nullfolgen. Lean beweist daraus `funext`: Funktionen mit denselben Werten sind gleich.
- `Classical.choice` wählt ein Element, ohne es zu berechnen. Mit den beiden anderen folgt daraus $P \vee \neg P$; was es benutzt, heißt `noncomputable`.
- Ob die drei zusammen widerspruchsfrei sind, prüft kein Kern – das muss man glauben, ebenso, dass der Kern richtig rechnet.

Konstruktiv oder klassisch?

`#print axioms` durchläuft einen Beweisterm samt allen Sätzen, die er benutzt, und zählt die Axiome darin auf:

Lean `code/Typentheorie.lean`

```
#print axioms Nat.zero_add
#print axioms Classical.em
#print axioms Function.cantor_surjective
```

```
'Nat.zero_add' does not depend on any axioms
'Classical.em' depends on axioms: [propext, Classical.choice, Quot.sound]
'Function.cantor_surjective' does not depend on any axioms
```

- $0 + n = n$ und der Satz von Cantor brauchen kein Axiom: Induktion ist eine Regel der Typentheorie, und das Diagonalargument ist konstruktiv.
- $P \vee \neg P$ (`Classical.em`) braucht alle drei. Ohne Axiome ist die Typentheorie *konstruktiv*: Ein Beweis von $P \vee \neg P$ müsste für jedes P entscheiden, welche Seite gilt. Mit den Axiomen folgt es (Satz von Diaconescu).

Live in Lean `code/Typentheorie.lean` – die drei `#print axioms`; Diaconescus Beweis in Lean folgt nach der Beweisirrelevanz

Vorlesung 2

Urteile und Regeln

Urteile und Kontexte

Nach welchen Regeln entscheidet sich, ob ein Ausdruck einen Typ hat?

Urteil

Eine Typentheorie wird durch *Urteile* beschrieben. Das wichtigste ist

$$\Gamma \vdash t : A \qquad \text{„Im Kontext } \Gamma \text{ hat der Term } t \text{ den Typ } A.\text{“}$$

Der *Kontext* ist eine Liste von Annahmen $\Gamma = x_1 : A_1,\ x_2 : A_2,\ \ldots,\ x_n : A_n$, in der jedes A_i ein Typ ist, der nur die vorherigen Variablen benutzt. Daneben gibt es das Urteil $\Gamma \vdash A \equiv B$ für definitorische Gleichheit.

- Zur Reihenfolge: In `meineId {`α `: Type u} (a :` α`)` ist der Kontext $\alpha : \mathrm{Type}\ u,\ a : \alpha$ – der Typ von a benutzt das α davor.

#check fragt nach einem Urteil

Lean code/Typentheorie.lean

```
/- Ein Urteil hat die Form 'Γ ⊢ t : A': Im Kontext 'Γ' hat der Term 't'
   den Typ 'A'. Genau danach fragt '#check'. -/
#check (3 : ℕ)
#check (fun x : ℕ => x + 1)
#check ℕ
#check Type

/- Der Kontext sind die Annahmen links vom Doppelpunkt. -/
example (x : ℕ) (y : ℕ) : ℕ := x + y
```

- #check fragt zu einem Term t: Für welches A gilt $\vdash t : A$, im leeren Kontext? Hier etwa $\vdash 3 : \mathbb{N}$ und $\vdash (\lambda x.\, x + 1) : \mathbb{N} \to \mathbb{N}$, ebenso $\vdash \mathbb{N} : \mathrm{Type}$ und $\vdash \mathrm{Type} : \mathrm{Type}\ 1$.
- Die letzte Zeile hat einen Kontext: die Annahmen `(x : ℕ) (y : ℕ)` links vom Doppelpunkt. Geprüft wird $x : \mathbb{N},\ y : \mathbb{N} \vdash x + y : \mathbb{N}$.

Die ersten Regeln: Variable und Universum

Die Regeln haben die Form „aus den Prämissen oben folgt die Konklusion unten“. Es sind nur wenige. Für die Universen steht dabei ein gemeinsamer Name, $\mathrm{Sort}\ u$:

$\mathrm{Sort}\ 0$	$\mathrm{Sort}\ 1$	$\mathrm{Sort}\ 2$	$\mathrm{Sort}\ 3$	$\cdots$
Prop	$\mathrm{Type} = \mathrm{Type}\ 0$	$\mathrm{Type}\ 1$	$\mathrm{Type}\ 2$	$\cdots$

So schreibt man die Regeln einmal hin, statt zweimal – für Aussagen und für Daten.

$$\frac{(x : A) \in \Gamma}{\Gamma \vdash x : A}\ (\textsc{Var}) \qquad \frac{}{\Gamma \vdash \mathrm{Sort}\ u : \mathrm{Sort}\ (u+1)}\ (\textsc{Sort})$$

- Eine Variable hat den Typ, unter dem sie im Kontext steht; Universen liegen jeweils eine Stufe höher.
- (Sort) mit $u = 0$ liefert $\vdash \mathrm{Prop} : \mathrm{Type}$, mit $u = 1$ liefert sie $\vdash \mathrm{Type} : \mathrm{Type}\ 1$.

Der abhängige Funktionstyp: drei Regeln

Eine Regel zum Bilden des Typs (Formation), eine zum Erzeugen seiner Elemente (Einführung), eine zum Anwenden (Beseitigung):

$$\frac{\Gamma \vdash A : \mathrm{Sort}\ u \qquad \Gamma,\, x : A \vdash B : \mathrm{Sort}\ v}{\Gamma \vdash (x : A) \to B : \mathrm{Sort}\ (\mathrm{imax}\ u\ v)}\ (\Pi\text{-Form})$$

$$\frac{\Gamma,\, x : A \vdash t : B}{\Gamma \vdash (\lambda x.\, t) : (x : A) \to B}\ (\lambda) \qquad \frac{\Gamma \vdash f : (x : A) \to B \qquad \Gamma \vdash a : A}{\Gamma \vdash f\ a : B[a/x]}\ (\mathrm{App})$$

- (λ): Wer unter der Annahme $x : A$ ein $t : B$ baut, hat eine Funktion. (App): Im Ergebnistyp wird a für x eingesetzt.
- Hängt B nicht von x ab, schreibt man kurz $A \to B$.
- Dazu kommen die Regeln der induktiven Typen: zu jedem seine Konstruktoren und sein Rekursor.

Wo die Impredikativität steckt

In (Π-Form) steht imax, nicht $\max$: $\mathrm{imax}\ u\ 0 = 0$, sonst $\mathrm{imax}\ u\ v = \max(u, v)$.

- **Über alle Aussagen:** $(p : \mathrm{Prop}) \to (p \to p)$. Hier ist $\mathrm{Prop} : \mathrm{Sort}\ 1$, also $u = 1$, und $p \to p : \mathrm{Sort}\ 0$, also $v = 0$:

  $$\mathrm{Sort}\ (\mathrm{imax}\ 1\ 0) \ = \ \mathrm{Sort}\ 0 \ = \ \mathrm{Prop}.$$

  Ein Funktionstyp mit Ziel in Prop landet wieder in Prop, ganz gleich, wie groß die Quelle ist.
- **Über alle Typen:** $(\alpha : \mathrm{Type}) \to (\alpha \to \alpha)$. Hier ist $\mathrm{Type} : \mathrm{Sort}\ 2$ und $\alpha \to \alpha : \mathrm{Sort}\ 1$:

  $$\mathrm{Sort}\ (\mathrm{imax}\ 2\ 1) \ = \ \mathrm{Sort}\ 2 \ = \ \mathrm{Type}\ 1$$ – eine Stufe höher.

- Deshalb darf man in Lean über *alle* Aussagen quantifizieren und bleibt trotzdem eine Aussage. Für Type gilt das nicht – sonst hätte man Girards Paradoxon.

Regeln für $\mathbb{N}$

Zu jeder Typbildung gehören vier Arten von Regeln: *Formation* (wie man den Typ bildet), *Einführung* (Elemente bauen), *Beseitigung* (Elemente benutzen) und *Berechnung* (wie beides zusammen rechnet). Für $\mathbb{N}$:

$$\underbrace{\frac{}{\Gamma \vdash \mathbb{N} : \mathrm{Type}}\ (\mathbb{N}\text{-Form})}_{\text{Formation}}$$

$$\underbrace{\frac{}{\Gamma \vdash \mathrm{zero} : \mathbb{N}}\ (\mathrm{zero}) \qquad \frac{\Gamma \vdash n : \mathbb{N}}{\Gamma \vdash \mathrm{succ}\ n : \mathbb{N}}\ (\mathrm{succ})}_{\text{Einführung}}$$

$$\underbrace{\frac{\begin{array}{c}\Gamma \vdash C : \mathbb{N} \to \mathrm{Sort}\ u \qquad \Gamma \vdash z : C\ \mathrm{zero} \\ \Gamma \vdash s : (k : \mathbb{N}) \to C\ k \to C\ (\mathrm{succ}\ k) \qquad \Gamma \vdash n : \mathbb{N}\end{array}}{\Gamma \vdash \mathrm{rec}\ z\ s\ n : C\ n}\ (\mathrm{rec})}_{\text{Beseitigung}}$$

- Die Namen: (zero) und (succ) heißen nach den Konstruktoren, der Null und dem Nachfolger; (rec) heißt nach dem *Rekursor* rec, in Lean `Nat.rec`.
- Berechnung, die ι-Regeln (der Rekursor trifft auf einen Konstruktor): $\mathrm{rec}\ z\ s\ \mathrm{zero} \equiv z$ und $\mathrm{rec}\ z\ s\ (\mathrm{succ}\ n) \equiv s\ n\ (\mathrm{rec}\ z\ s\ n)$.

Der Rekursor: Rekursion und Induktion

$\mathrm{rec}\ z\ s\ n$ macht aus einer Zahl n ein Element von $C\ n$. Die Prämissen von (rec) sagen, was man dafür angibt:

- $C : \mathbb{N} \to \mathrm{Sort}\ u$, das *Motiv*: für jedes n der Typ des Ergebnisses;
- $z : C\ \mathrm{zero}$: das Ergebnis für zero;
- $s : (k : \mathbb{N}) \to C\ k \to C\ (\mathrm{succ}\ k)$, der Schritt: Aus k und dem Ergebnis r für k macht $s\ k\ r$ das Ergebnis für $\mathrm{succ}\ k$.

Nach den ι-Regeln ist $f := \mathrm{rec}\ z\ s$ also die Funktion mit $f\ 0 \equiv z$ und $f\ (k+1) \equiv s\ k\ (f\ k)$ – eine Definition durch Rekursion.

Rekursion, C landet in Type: Mit $C\ n := \mathbb{N}$, $z := 0$ und $s\ k\ r := r + 2$ ist $f\ 0 \equiv 0$ und $f\ (k+1) \equiv f\ k + 2$, also $f\ n = 2n$:

$f\ 2 \equiv f\ 1{+}2 \equiv (f\ 0{+}2){+}2 \equiv (0{+}2){+}2.$

Induktion, C landet in Prop: Mit $C\ n := (0 + n = n)$ ist z ein Beweis des Anfangs $0 + 0 = 0$, und $s\ k\ r$ macht aus dem Beweis r für k einen für $k+1$. So beweist $f\ n$ die Aussage für jedes n.

Live in Lean `code/Typentheorie.lean` – `#check @Nat.rec`: die Regel (rec) als Konstante

Pause

Danach: Regeln für Paare, Summen und Gleichheit, Herleitungen –
dann, mit Curry–Howard, warum ein Term ein Beweis ist

Vorlesung 2

Paare, Summen, Gleichheit und Herleitungen

Regeln für Paare und Summen

Formation: Aus $A : \mathrm{Type}\ u$ und $B : \mathrm{Type}\ v$ folgt $A \times B,\ A \oplus B : \mathrm{Type}\ (\max\ u\ v)$.

$$\underbrace{\frac{\Gamma \vdash a : A \quad \Gamma \vdash b : B}{\Gamma \vdash (a, b) : A \times B}\ (\textsc{Paar})}_{\text{Einführung}} \qquad \underbrace{\frac{\Gamma \vdash p : A \times B}{\Gamma \vdash p.1 : A}\ (\textsc{fst}) \qquad \frac{\Gamma \vdash p : A \times B}{\Gamma \vdash p.2 : B}\ (\textsc{snd})}_{\text{Beseitigung}}$$

$$\underbrace{\frac{\Gamma \vdash a : A}{\Gamma \vdash \mathrm{inl}\ a : A \oplus B}\ (\textsc{inl}) \qquad \frac{\Gamma \vdash b : B}{\Gamma \vdash \mathrm{inr}\ b : A \oplus B}\ (\textsc{inr})}_{\text{Einführung}}$$

$$\underbrace{\frac{\Gamma \vdash s : A \oplus B \quad \Gamma \vdash f : A \to C \quad \Gamma \vdash g : B \to C}{\Gamma \vdash \mathrm{elim}\ f\ g\ s : C}\ (\textsc{Fall})}_{\text{Beseitigung}} \qquad \underbrace{\begin{aligned}\mathrm{elim}\ f\ g\ (\mathrm{inl}\ a) &\equiv f\ a\\ \mathrm{elim}\ f\ g\ (\mathrm{inr}\ b) &\equiv g\ b\end{aligned}}_{\text{Berechnung}}$$

- Die Berechnungsregeln *definieren* elim: Ist s mit inl links eingebettet, wird f angewandt, mit inr rechts, g. Für Paare gilt $(a, b).1 \equiv a$, $(a, b).2 \equiv b$.
- Für $\wedge$ und $\vee$ gelten dieselben Regeln mit Prop statt Type, bei $\vee$ aber nur mit $C : \mathrm{Prop}$. In Lean: `Prod`, `Sum`, `And`, `Or`.

Regeln für die Gleichheit

Auch die Gleichheit ist ein induktiver Typ, mit einem einzigen Konstruktor, rfl:

$$\underbrace{\frac{\Gamma \vdash a : A \quad \Gamma \vdash b : A}{\Gamma \vdash a = b : \mathrm{Prop}}\ (=\text{-Form})}_{\text{Formation}} \qquad \underbrace{\frac{\Gamma \vdash a : A}{\Gamma \vdash \mathrm{rfl} : a = a}\ (\text{rfl})}_{\text{Einführung}}$$

$$\underbrace{\frac{\Gamma \vdash P : A \to \mathrm{Sort}\ u \quad \Gamma \vdash h : a = b \quad \Gamma \vdash t : P\ a}{\Gamma \vdash h \blacktriangleright t : P\ b}\ (\text{Transport})}_{\text{Beseitigung}} \qquad \underbrace{\mathrm{rfl} \blacktriangleright t \ \equiv\ t}_{\text{Berechnung}}$$

- (rfl) steht für Reflexivität. Das Dreieck ist Leans Schreibweise `h ▶ t` (Eingabe `\t`), gelesen „t entlang h“: Es bringt $t : P\ a$ mit $h : a = b$ nach $P\ b$.
- Definiert ist ▶ wie elim: (Transport) sagt, welchen Typ $h \blacktriangleright t$ hat, die Berechnungsregel, was es tut. In Lean steht dahinter der Rekursor `Eq.rec`, erzeugt aus dem Konstruktor `rfl`.
- Daraus folgen Symmetrie, Transitivität und Kongruenz. Die Symmetrie etwa: Mit $P\ x := (x = a)$ ist $\mathrm{rfl} : P\ a$, also $h \blacktriangleright \mathrm{rfl} : P\ b$, und $P\ b$ ist $b = a$.
- Nicht aber `funext` und `propext`: Dafür braucht es die Axiome von vorhin.

Eine Herleitung, rückwärts gesucht

Ein erstes Beispiel. Der Kontext $\Gamma_0 := A : \mathrm{Prop},\ B : \mathrm{Prop}$ sagt nur, dass A und B Aussagen sind. Gesucht ist eine Herleitung von

$$\Gamma_0 \vdash \lambda a.\, \lambda b.\, a \ :\ A \to (B \to A)$$

1. Eine Abstraktion, also (λ). Prämisse: $\Gamma_0,\ a : A \vdash \lambda b.\, a : B \to A$.
2. Noch einmal (λ). Prämisse: $\Gamma_0,\ a : A,\ b : B \vdash a : A$.
3. (Var): $(a : A)$ steht im Kontext. Fertig.

$$\dfrac{\dfrac{\dfrac{(a : A) \in (\Gamma_0,\ a : A,\ b : B)}{\Gamma_0,\ a : A,\ b : B \vdash a : A}\ (\textsc{Var})}{\Gamma_0,\ a : A \vdash \lambda b.\, a : B \to A}\ (\lambda)}{\Gamma_0 \vdash \lambda a.\, \lambda b.\, a : A \to (B \to A)}\ (\lambda)$$

Ein Typcheck mit Herleitungsbaum

Gilt $\vdash \lambda f.\, \lambda x.\, f\,(f\,x) : (\mathbb{N} \to \mathbb{N}) \to (\mathbb{N} \to \mathbb{N})$? Man sucht die Herleitung von unten nach oben; die Form des Terms bestimmt die Regel: Abstraktion (λ), Anwendung (App), Variable (Var).

$$\dfrac{\dfrac{\dfrac{\dfrac{}{\Gamma \vdash f : \mathbb{N} \to \mathbb{N}}\,(\textsc{Var}) \quad \dfrac{\dfrac{}{\Gamma \vdash f : \mathbb{N} \to \mathbb{N}}\,(\textsc{Var}) \quad \dfrac{}{\Gamma \vdash x : \mathbb{N}}\,(\textsc{Var})}{\Gamma \vdash f\,x : \mathbb{N}}\,(\textsc{App})}{\Gamma \vdash f\,(f\,x) : \mathbb{N}}\,(\textsc{App})}{\Gamma_1 \vdash \lambda x.\, f\,(f\,x) : \mathbb{N} \to \mathbb{N}}\,(\lambda)}{\vdash \lambda f.\, \lambda x.\, f\,(f\,x) : (\mathbb{N} \to \mathbb{N}) \to (\mathbb{N} \to \mathbb{N})}\,(\lambda)$$

- Dabei ist $\Gamma_1 := f : \mathbb{N} \to \mathbb{N}$ und $\Gamma := \Gamma_1,\ x : \mathbb{N}$. Bei (App) nennt das Ziel nur den Ergebnistyp; A liest man am Typ der Funktion f ab.
- Über den Blättern (Var) fehlt nur die erfüllte Bedingung $(x : A) \in \Gamma$.

Derselbe Typcheck in Lean

Lean code/Typentheorie.lean

```
example : (ℕ → ℕ) → ℕ → ℕ := by
  intro f      -- (λ):    f : ℕ → ℕ ⊢ ℕ → ℕ
  intro x      -- (λ):    f : ℕ → ℕ, x : ℕ ⊢ ℕ
  apply f      -- (App):  Funktion f; offen bleibt das Argument, ⊢ ℕ
  apply f      -- (App):  noch einmal f; offen bleibt ⊢ ℕ
  exact x      -- (Var):  x : ℕ steht im Kontext
```

- Lean nennt die Regeln nicht beim Namen. Im Taktikmodus wendet aber jede Taktik eine Regel von unten nach oben an: `intro` ist (λ), `apply f` ist (App) mit der Funktion f aus dem Kontext, `exact x` ist (Var).
- Nach jeder Zeile zeigt das Infofenster das offene Urteil, also einen Knoten des Baums. Im Termmodus steht der fertige Term direkt da:

Lean code/Typentheorie.lean

```
/- Lean findet die Herleitung von ⊢ fun f x => f (f x) : (ℕ → ℕ) → ℕ → ℕ
   selbst. -/
example : (ℕ → ℕ) → ℕ → ℕ := fun f x => f (f x)
#check fun (f : ℕ → ℕ) (x : ℕ) => f (f x)
```

Live in Lean `code/Typentheorie.lean` – die Taktiken nacheinander anklicken, das Ziel lesen

Anwenden heißt einsetzen

Bei einem echt abhängigen Typ steckt in (App) die Substitution $B[a/x]$:

Lean code/Typentheorie.lean

```
/-- Ein **abhaengiger** Funktionstyp: der Ergebnistyp haengt vom Argument ab. -/
def diagonale : (n : ℕ) → (Fin n → Fin n) := fun _ i => i
```

$$\frac{\vdash \texttt{diagonale} : (n : \mathbb{N}) \to (\mathrm{Fin}\ n \to \mathrm{Fin}\ n) \qquad \vdash 3 : \mathbb{N}}{\vdash \texttt{diagonale}\ 3 : (\mathrm{Fin}\ n \to \mathrm{Fin}\ n)[3/n] \;=\; \mathrm{Fin}\ 3 \to \mathrm{Fin}\ 3}\ (\textsc{App})$$

- Der Ergebnistyp hängt vom Argument ab; (App) setzt den Wert 3 für n ein.
- Wendet man `Vektor.head` auf `Vektor.nil` an, scheitert die zweite Prämisse. Das Argument hat den Typ `Vektor` α `0`, und kein n macht daraus `Vektor` α `(n + 1)`.

Die Konversionsregel

Die wichtigste Regel von allen:

$$\frac{\Gamma \vdash t : A \qquad \Gamma \vdash A \equiv B}{\Gamma \vdash t : B}\ (\textsc{Conv})$$

Sie verbindet Rechnen und Typisierung: Weil Typen von Werten abhängen dürfen, stehen in Typen Rechnungen – und es kommt auf das Ergebnis an, nicht auf die Schreibweise.

Lean `code/Typentheorie.lean`

```
example (v : Vektor ℕ (2 + 1)) : Vektor ℕ 3 := v
```

$$\frac{\Gamma \vdash v : \mathrm{Vektor}\ \mathbb{N}\ (2+1) \qquad \Gamma \vdash \mathrm{Vektor}\ \mathbb{N}\ (2+1) \equiv \mathrm{Vektor}\ \mathbb{N}\ 3}{\Gamma \vdash v : \mathrm{Vektor}\ \mathbb{N}\ 3}\ (\textsc{Conv})$$

Ohne die Regel wäre `Vektor ℕ (2 + 1)` ein anderer Typ als `Vektor ℕ 3`. Der Preis: Der Typchecker muss rechnen können.

Vorlesung 2

Die Curry–Howard-Korrespondenz

Aussagen sind Typen, Beweise sind Terme

Logik und Typentheorie sind nicht zwei Dinge, sondern eines:

Logik	**Typentheorie**	**in Lean**
Aussage A	Typ A	`A : Prop`
Beweis von A	Term vom Typ A	`h : A`
$A \to B$	Funktionstyp	`A → B`
$A \wedge B$	Produkt	`A ∧ B`
$A \vee B$	Summe	`A ∨ B`
$\neg A$	Funktion in den leeren Typ	`A → False`
$\forall x, P(x)$	abhängiger Funktionstyp	`∀ x, P x`
$\exists x, P(x)$	abhängiges Paar	`∃ x, P x`

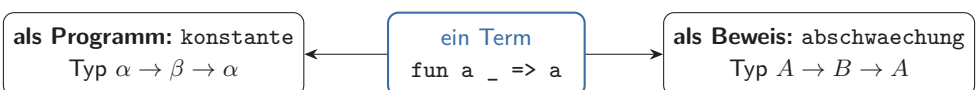
Ein Beweis ist ein Programm: Einen Satz beweisen heißt, einen Term des passenden Typs angeben; einen Beweis prüfen heißt, den Term typchecken. Warum ein Term ein Beweis ist, zeigen die Regeln von eben.

Derselbe Term, zwei Lesarten

Lean code/Typentheorie.lean

```
/- Derselbe Term, zwei Lesarten: einmal als Programm, einmal als Beweis. -/
def konstante {α β : Type} : α → β → α := fun a _ => a
theorem abschwaechung {A B : Prop} : A → B → A := fun a _ => a
```

- `konstante` und `abschwaechung` haben denselben Term `fun a _ => a`.
- Einmal liest man ihn als Funktion, die das erste Argument zurückgibt. Einmal als Beweis, dass aus A und B wieder A folgt.



Derselbe Term in fünf Systemen

Der Term `fun a _ => a` mit dem Typ $A \to B \to A$ hat in fünf Systemen eine eigene Gestalt und Rolle:

System	Gestalt	Rolle
Hilbert-Kalkül	(K): $\alpha \to (\beta \to \alpha)$	Axiom
natürliches Schließen	zweimal $\to$I, Annahme ψ leer entlassen	hergeleitet
λ-Kalkül	$\mathrm{wahr} := \lambda x.\, \lambda y.\, x$	Wahrheitswert
Haskell	`const :: a -> b -> a`	Programm
Lean	`abschwaechung`, Typ $A \to B \to A$	Beweis

- Kein Zufall, sondern Curry–Howard: Formeln sind Typen, Herleitungen sind Terme.
- Ebenso die Identität: Mit $K := \lambda x.\, \lambda y.\, x$ und $S := \lambda f.\, \lambda g.\, \lambda x.\, f\, x\, (g\, x)$ ist $S\, K\, K\, x = K\, x\, (K\, x) = x$ – die Hilbert-Herleitung von $\varphi \to \varphi$ aus den Axiomen (S) und (K), als Term geschrieben.

Die Junktoren als Typkonstruktionen

Lean code/Typentheorie.lean

```
/- Die Junktoren sind Typkonstruktionen: -/
example (P : Prop) : (¬ P) = (P → False) := rfl          -- Negation = Funktion nach False
example (A B : Prop) (h : A ∧ B) : A := h.1               -- Konjunktion = Produkt
example (A B : Prop) (h : A ∨ B) : B ∨ A :=               -- Disjunktion = Summe
  h.elim Or.inr Or.inl
example (p : ℕ → Prop) (h : ∃ n, p n) : ∃ m, p m := h    -- Existenz = abh. Paar

/- Ein Existenzbeweis *ist* ein Paar aus Zeuge und Nachweis. -/
example : ∃ n : ℕ, 3 < n := ⟨4, by norm_num⟩
```

- $\neg P$ *ist* per Definition eine Funktion nach `False`: `rfl` genügt.
- $A \wedge B$ ist ein Produkt: `h` ist ein Paar, `h.1` seine erste Komponente.
- $A \vee B$ ist eine Summe: `h.elim Or.inr Or.inl` wendet (Fall) an, mit `h` vorn. Ein Beweis von A geht nach rechts, einer von B nach links.
- $\exists n,\ p\ n$ und $\exists m,\ p\ m$ sind derselbe Typ – nur die gebundene Variable heißt anders; also genügt `h`.
- Ein Existenzbeweis *ist* ein Paar: $\langle$`4, by norm_num`$\rangle$, Zeuge 4, Nachweis $3 < 4$.

Terme weglassen: natürliches Schließen

Typentheorie (B ohne x)

$$\frac{\Gamma,\, x : A \vdash t : B}{\Gamma \vdash \lambda x.\, t : A \to B}\ (\lambda)$$

$$\frac{\Gamma \vdash f : A \to B \quad \Gamma \vdash a : A}{\Gamma \vdash f\, a : B}\ (\textsc{App})$$

Natürliches Schließen

$$\frac{\Sigma,\, \varphi \vdash \psi}{\Sigma \vdash \varphi \to \psi}\ (\to\text{I})$$

$$\frac{\Sigma \vdash \varphi \to \psi \quad \Sigma \vdash \varphi}{\Sigma \vdash \psi}\ (\to\text{E})$$

- Streicht man die Terme, bleiben Gentzens Regeln für den Pfeil: Der Kontext sind die offenen Annahmen, (λ) entlässt eine wie $\to$I, (App) ist $\to$E, also der modus ponens.
- Ebenso: (Var) ist eine Annahme, (Paar) $\wedge$I, (fst), (snd) $\wedge$E, (inl), (inr) $\vee$I, (Fall) $\vee$E mit den Fallbeweisen als Funktionen; hängt B von x ab, sind (λ), (App) $\forall$I, $\forall$E.
- Ohne Terme ist die rückwärts gesuchte Herleitung von $\lambda a.\, \lambda b.\, a$ Gentzens Beweis von (K):

$$\dfrac{\dfrac{[\varphi]^1}{\psi \to \varphi}\ (\to\text{I},\, 2)}{\varphi \to (\psi \to \varphi)}\ (\to\text{I},\, 1)$$

Warum ein Term ein Beweis ist

Ein Beispiel mit $\wedge$ und $\to$; jeder Knoten trägt seinen Term:

$$\dfrac{\dfrac{\dfrac{[h : A \wedge (A \to B)]^1}{h.2 : A \to B}\,(\wedge\mathrm{E}) \qquad \dfrac{[h : A \wedge (A \to B)]^1}{h.1 : A}\,(\wedge\mathrm{E})}{h.2\ h.1 : B}\,(\to\mathrm{E})}{\lambda h.\, h.2\ h.1 : A \wedge (A \to B) \to B}\,(\to\mathrm{I},\ 1)$$

- Ohne die Terme steht da Gentzens Beweis: Annahme, zweimal $\wedge$E, modus ponens, Annahme 1 entlassen. Ohne die Formeln wächst von oben nach unten der Term $\lambda h.\, h.2\ h.1$; der Name h spielt die Rolle der Nummer 1.
- Umgekehrt legt der Term mit seinem Typ den Baum fest: Seine äußere Form bestimmt die unterste Regel, seine Teilterme die Teilbäume darüber. Ein Term ist ein Beweis, in eine Zeile geschrieben. Der Typcheck baut den Baum auf und prüft jeden Schritt: Typchecken ist Beweisprüfen.
- Das passt zur Deutung nach Brouwer, Heyting und Kolmogorow (Skript 2.5): Ein Beweis von $A \to B$ verwandelt jeden Beweis von A in einen von B – er ist eine Funktion.

Live in Lean `code/Typentheorie.lean` – `abtrennung` als Term und mit Taktiken, dazu `#print`

Induktion ist ein Term

Mit einem Motiv in `Prop` ist (rec) die Induktion: Für $C\ n := \varphi(n)$ ist z der Anfang und s der Schritt.

$$\frac{z : \varphi(0) \qquad s : (k : \mathbb{N}) \to \varphi(k) \to \varphi(k+1)}{\mathrm{rec}\ z\ s\ n : \varphi(n)}\ (\textsc{rec})$$

Ohne Terme ist das die Induktionsregel. Für $0 + n = n$ bei `MeinNat`:

Lean code/Typentheorie.lean

```
/- Induktion als Term: der Rekursor mit einem Motiv in Prop. Derselbe Satz wie
   ‘MeinNat.zero_add‘ in 3.2, dort mit der Taktik ‘induction‘. -/
theorem MeinNat.zero_add_term (n : MeinNat) : MeinNat.zero.add n = n :=
  MeinNat.rec (motive := fun n => MeinNat.zero.add n = n)
    rfl                                  -- Anfang: 0 + 0 = 0
    (fun _ ih => congrArg MeinNat.succ ih)  -- Schritt: aus 0 + k = k wird 0 + (k+1) = k+1
    n
```

- Der Schritt ist eine Funktion: Aus einem Beweis `ih` für k macht `congrArg MeinNat.succ` einen für $k+1$, wie es die Deutung nach Brouwer, Heyting und Kolmogorow verlangt.
- Für eine feste Zahl rechnet der Term aus: $\mathrm{rec}\ z\ s\ 3 \rightsquigarrow s\ 2\ (s\ 1\ (s\ 0\ z))$. Die Taktik `induction` auf „Rechnen oder Induzieren“ baut genau so einen Term.

Warum `Prop` von `Type` getrennt ist

Wenn Beweise Terme sind – wozu dann die eigene Stufe `Prop`? Weil Beweise etwas haben, was Daten nicht haben: Sie sind irrelevant.

Satz (Beweisirrelevanz)

Sind h_1, h_2 Beweise derselben Aussage $P : \mathrm{Prop}$, so gilt $h_1 = h_2$, und zwar definitorisch.

Lean code/Typentheorie.lean

```
example (P : Prop) (h₁ h₂ : P) : h₁ = h₂ := rfl
```

- Läge $\mathbb{N}$ in `Prop` statt in `Type`, so wären `0` und `1` definitorisch gleich – und darauf, dass sie verschieden sind, beruht jede Rechnung.

Beweiseindeutigkeit

`Prop`: der Unterschied in einem Satz

Von einem Beweis zählt nur, *dass* es ihn gibt, nicht *welcher* er ist.

Eine Aussage kann man auf viele Arten beweisen – nach der Beweisirrelevanz sind alle diese Beweise *definitorisch* gleich, auch sichtbar verschieden hergestellte:

Lean `code/Typentheorie.lean`

```
example : 0 < 2 := by norm_num
example : 0 < 2 := by omega
example (h₁ h₂ : 0 < 2) : h₁ = h₂ := rfl
```

- Mit `norm_num` und mit `omega` entstehen zwei sichtbar verschieden hergestellte Beweise von `0 < 2` – und doch sind je zwei Beweise dieser Aussage gleich.

UIP – und Daten sind anders

Lean code/Typentheorie.lean

```
/- Das greift weiter, als es zunaechst aussieht: ‘a = b‘ ist selbst eine
   ‘Prop‘. Also sind auch je zwei *Gleichheitsbeweise* gleich -- das nennt
   man UIP, uniqueness of identity proofs. -/
example {α : Type} {a b : α} (p q : a = b) : p = q := rfl

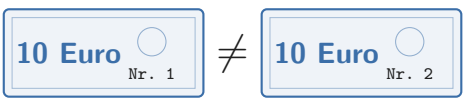
/- Bei Daten ist es umgekehrt: verschiedene Werte bleiben verschieden. -/
example : (0 : ℕ) ≠ 1 := by decide
example (b : Bool) : b = true ∨ b = false := by cases b <;> simp
```

- `a = b` ist selbst eine `Prop`. Also sind auch je zwei *Gleichheitsbeweise* gleich: **UIP**, *uniqueness of identity proofs*.
- Bei Daten ist es umgekehrt: `decide` rechnet $0 \neq 1$ einfach aus; `cases b <;> simp` geht die beiden Fälle durch – jeder Wert von `Bool` ist `true` oder `false`, und es zählt, welcher.
- Genau deshalb trennt Lean die beiden Stufen: `Prop` für Dinge, bei denen nur die Existenz zählt, `Type` für Dinge, die man unterscheiden können muss.

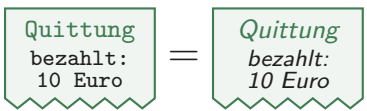
Quittung und Geldschein

Zwei Geldscheine über zehn Euro sind zwei Scheine. Man kann sie auseinanderhalten, sie haben verschiedene Seriennummern, und es macht einen Unterschied, welchen man hat.

Zwei Quittungen über dieselbe Zahlung sind dagegen nur *ein* Vorgang. Ob der Beleg getippt oder handgeschrieben ist, spielt keine Rolle – bezahlt ist bezahlt. Genau so verhält sich `Prop`.



zwei Scheine, unterscheidbar – wie Daten in `Type`

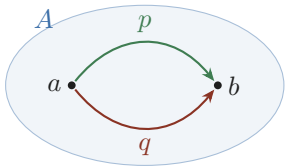


getippt handgeschrieben

ein Vorgang – wie Beweise in `Prop`

Der andere Weg: Univalenz und HoTT

- Beweiseindeutigkeit ist eine *Entwurfsentscheidung*, keine Notwendigkeit.
- *Homotopische Typentheorie* (HoTT, nach Voevodsky): $a = b$ ist ein Typ, der Struktur haben darf – es kann mehrere, verschiedene Gleichheiten geben.
- *Proposition* heißt dort ein Typ, in dem je zwei Elemente gleich sind – Eigenschaft statt Vorschrift.



Typ als *Raum*, Gleichheit als *Weg*.
HoTT: $p, q : a = b$ dürfen verschieden sein.
Lean: stets $p = q$ (UIP).

Univalenzaxiom: $(A \simeq B) \simeq (A = B)$ – äquivalente Typen *sind* gleich. Isomorphe Strukturen identifizieren, was Mathematiker ohnehin tun, wird Rechenregel statt Handarbeit.

Beides zusammen geht nicht

1. In Lean ist $a = b$ eine `Prop`. Also sind je zwei Gleichheitsbeweise gleich (UIP).
2. $\mathrm{Bool} \simeq \mathrm{Bool}$ hat zwei verschiedene Elemente: die Identität und die Negation. (Sie unterscheiden sich schon an der Stelle true.)
3. Univalenz ordnet Äquivalenzen umkehrbar eindeutig Gleichheiten zu: $(\mathrm{Bool} \simeq \mathrm{Bool}) \simeq (\mathrm{Bool} = \mathrm{Bool})$. Also hätte $\mathrm{Bool} = \mathrm{Bool}$ zwei verschiedene Elemente.
4. Das widerspricht Schritt 1.

Mathlib. Mathlib erledigt den Transport entlang von Isomorphien deshalb von Hand, über `Equiv` und passende Lemmas. Das ist Arbeit, die Univalenz geschenkt gäbe, und einer der meistdiskutierten Nachteile dieses Entwurfs. In *Cubical Agda* hat Univalenz sogar Rechenverhalten.

Was das praktisch bedeutet

- **Beweise löschen.** Beim Übersetzen darf man alle Beweise weglassen: Kein Programm kann sie unterscheiden, also trägt keines Information über sie.
- **Keine Sorge um das Welche.** Man muss nie fragen, *welchen* Beweis man hat – eine Sorge weniger, die in anderen Systemen echte Arbeit macht.
- **Der Preis.** Ein Beweis von $\exists$ `n, p n` ist zwar ein Paar, doch das n lässt sich nicht herausrechnen: Zwei Beweise mit verschiedenen Zeugen sind ja gleich. Wer den Zeugen braucht, formuliert die Aussage in `Type` statt in `Prop`.

Live in Lean `code/Typentheorie.lean` – `tertium_non_datur`, Diaconescus Beweis: $\exists$ als Paar, Fallunterscheidung über $\vee$; dank der Beweisirrelevanz wählt `Classical.choose` aus gleichen Prädikaten dasselbe Element

Warum man den Zeugen nicht herausrechnen kann

Herausrechnen hieße: eine Funktion $\mathrm{zeuge} : (\exists n,\ p\ n) \to \mathbb{N}$ mit $\mathrm{zeuge}\,\langle n, h\rangle \equiv n$, wie (fst) bei Paaren: $(a, b).1 \equiv a$. Angenommen, es gäbe sie. Für $p\ n := (n < 2)$ sind $\langle 0, _\rangle$ und $\langle 1, _\rangle$ Beweise derselben Aussage, nach der Beweisirrelevanz also gleich. Dann wäre

$$0 \ \equiv\ \mathrm{zeuge}\,\langle 0, _\rangle \ \equiv\ \mathrm{zeuge}\,\langle 1, _\rangle \ \equiv\ 1.$$

Lean code/Typentheorie.lean

```
/- Zwei Beweise mit verschiedenen Zeugen sind gleich (Beweisirrelevanz): -/
example : (⟨0, by decide⟩ : ∃ n : ℕ, n < 2) = ⟨1, by decide⟩ := rfl
/- Den Zeugen herausrechnen lehnt Lean ab, `Exists.casesOn` beseitigt nur in Prop:
def zeuge (h : ∃ n : ℕ, n < 2) : ℕ := match h with | ⟨n, _⟩ => n -/
/- In `Type` ist das Paar Daten, und `.1` rechnet den Zeugen aus: -/
example : (⟨0, by decide⟩ : {n : ℕ // n < 2}).1 = 0 := rfl
```

- Deshalb beseitigt Lean $\exists$ wie $\vee$ nur in `Prop`: Aus einem Existenzbeweis folgt nur wieder eine Aussage. `Classical.choose` wählt zwar ein n, aber nicht berechenbar – und für alle Beweise dasselbe.
- Wer den Zeugen braucht, nimmt `{n // p n}` in `Type`: Dort sind $\langle 0, _\rangle$ und $\langle 1, _\rangle$ verschieden, und `.1` rechnet den Zeugen aus.

Vorlesung 2

Rechnen, Prüfen, Vertrauen

Woher die definitorische Gleichheit kommt

$A \equiv B$ ist kein Grundbegriff, sondern entsteht aus Reduktionsregeln:

Regel	was reduziert wird	Beispiel
β	Anwendung einer Abstraktion	$(\lambda x.\, t)\, a \rightsquigarrow t[a/x]$
δ	Auffalten einer Definition	`zwei` $\rightsquigarrow 2$
ι	Rekursor auf Konstruktor	$\mathrm{rec}\ \ldots\ (n{+}1) \rightsquigarrow \ldots$
ζ	`let`-Bindung einsetzen	$\mathbf{let}\ x := a;\ t \rightsquigarrow t[a/x]$
η	Funktionen ausdehnen	$f \equiv \lambda x.\, f\, x$

Dazu kommt die Beweisirrelevanz: Je zwei Beweise derselben Aussage sind definitorisch gleich. Alle diese Gleichheiten prüft Lean durch Ausrechnen.

- Ein Zahlenbeispiel zu ζ: $\mathbf{let}\ x := 2;\ x + x \rightsquigarrow 2 + 2$, ausgerechnet 4.

Reduktionen in Lean

Lean code/Typentheorie.lean

```
/- beta: eine Anwendung wird eingesetzt. -/
example : (fun x : ℕ => x + 1) 2 = 3 := rfl

/- delta: eine Definition wird aufgefaltet. -/
def zwei : ℕ := 2
example : zwei + zwei = 4 := rfl

/- iota: eine Fallunterscheidung trifft auf einen Konstruktor und rechnet. -/
def doppelt : ℕ → ℕ
  | 0 => 0
  | k + 1 => doppelt k + 2
example : doppelt 3 = 6 := rfl

/- eta: jede Funktion ist ihre eigene Abstraktion. -/
example (f : ℕ → ℕ) : f = fun x => f x := rfl
```

- β: $(\lambda x.\, x + 1)\, 2 \rightsquigarrow 2 + 1$. δ: `zwei` wird zu 2 aufgefaltet. η: f und $\lambda x.\, f\, x$ sind definitorisch gleich.
- ι: `doppelt` ist f von der Folie zum Rekursor: $f\ 3 \rightsquigarrow f\ 2 + 2 \rightsquigarrow \cdots \rightsquigarrow ((0 + 2) + 2) + 2$, ausgerechnet 6.

Warum der Typcheck entscheidbar ist

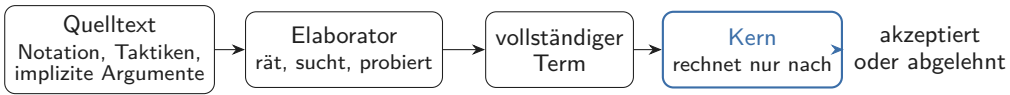
- Die Reduktion ist *konfluent* (die Reihenfolge der Schritte ist egal) und *stark normalisierend* (sie endet immer). Also hat jeder Term eine eindeutige Normalform, und $A \equiv B$ entscheidet man, indem man beide ausrechnet und vergleicht.
- Ohne Typen ginge das nicht: $(\lambda x.\, x\, x)\,(\lambda x.\, x\, x)$ rechnet ewig.
- Entscheidbar heißt nicht billig. Normalformen können riesig werden; Lean rechnet nur so weit wie nötig und vergleicht dann Stück für Stück.
- Der Typchecker arbeitet *bidirektional*: Mal rechnet er den Typ aus dem Term heraus, mal prüft er gegen einen erwarteten Typ. Deshalb braucht `fun x => x` eine Typangabe, `fun x : ℕ => x + 1` nicht.

Ausführlich

Reduktion, definitorische Gleichheit, Typinferenz: *Type Checking in Lean 4*, `https://ammkrn.github.io/type_checking_in_lean4/`

Elaborator und Kern

Was man schreibt, ist nicht, was geprüft wird: Der *Elaborator* füllt implizite Argumente, Instanzen und Koerzierungen ein und führt Taktiken aus. Erst der fertige Term geht an den *Kern*, und der kann nur eines: die Regeln anwenden.



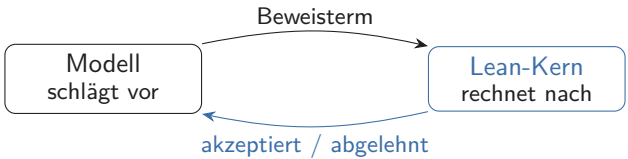
Ein Fehler im Elaborator lässt höchstens einen richtigen Beweis scheitern. Ein falscher käme nur durch einen Fehler im Kern durch – und der ist klein genug, um ihn zu überblicken. Deshalb kann man einem Beweisassistenten trauen.

Ausführlich

The Lean Language Reference, Kapitel „Elaboration and Compilation“,
`https://lean-lang.org/doc/reference/latest/Elaboration-and-Compilation/`

Beweise, die eine Maschine geschrieben hat

- Sprachmodelle schlagen Beweise vor; die falschen sehen den guten zum Verwechseln ähnlich.
- Für den Kern ist das ohne Belang. Ob Mensch, Modell oder Zufallsgenerator: Er wendet die Regeln an, und entweder sie passen oder nicht.



Hat der Kern einen maschinell gefundenen Beweis akzeptiert, muss ein Mensch nur noch die *Aussage* prüfen: ob der bewiesene Satz der gemeinte ist. Das ist viel weniger, als einen Beweis nachzuvollziehen – und es lässt sich nicht delegieren.

Vorlesung 2

Term- und Taktikmodus

Zwei Arten, einen Beweis anzugeben

Lean code/Typentheorie.lean

```
/-- Termmodus: der Beweis wird direkt hingeschrieben. -/
theorem symm_term (a b : ℕ) (h : a = b) : b = a := h.symm

/-- Taktikmodus: der Beweisterm wird schrittweise konstruiert.
Auch hier entsteht ein Kern-Term, nur ein anderer. -/
theorem symm_taktik (a b : ℕ) (h : a = b) : b = a := by
  rw [h]

example (a b : ℕ) (h : a = b) : b = a := by
  subst h
  rfl
```

- **Signatur:** gegeben `a b : ℕ` und `h : a = b`, gesucht ein Term vom Typ $b = a$.
- **Termmodus:** `h.symm`, die Symmetrie angewandt auf `h`, *ist* dieser Term.
- **Taktikmodus** (nach `by`): `rw [h]` macht das Ziel $b = a$ zu $b = b$, schließt mit `rfl`.
- `subst h` setzt die Gleichung ein, `rfl` erledigt den Rest.

Taktiken sind Bauanleitungen: Heraus kommt ein Term, den derselbe Kern prüft – nicht wörtlich derselbe wie `h.symm`, aber definitorisch gleich (Beweisirrelevanz).

Zusammenfassung

- Lean lesen und schreiben: `#check`, `#eval`, `def`, `fun`; Beweise als Term oder nach `by` mit Taktiken.
- Jeder Term hat genau einen Typ; $t : T$ ist Syntax, keine Behauptung. Die Universenhierarchie ist nötig (Girard) – das Gegenstück zu Cantor.
- Abhängige Typen schreiben Voraussetzungen in den Typ; induktive Typen liefern Rekursion und Induktion.
- Definitorische Gleichheit entsteht durch Rechnen, propositionale braucht einen Beweis. Darüber hinaus braucht Mathlib nur drei Axiome.
- Wenige Regeln – Var, Sort, Π-Form, λ, App, Conv und die der induktiven Typen. Ein Typcheck sucht die Herleitung von unten nach oben.
- Curry–Howard: Aussagen sind Typen, Beweise sind Terme; ohne Terme sind Einführung und Beseitigung Gentzens Regeln. Prüfen heißt typchecken.
- `Prop` ist beweisirrelevant: Je zwei Beweise sind definitorisch gleich, auch Gleichheitsbeweise (UIP). Daten in `Type` nicht.
- Term- und Taktikmodus liefern beide einen Term, den nur der Kern prüft.

Nächstes Mal: Mathematik in Lean

- **Mengen in Lean** (3.6): Mengen sind Prädikate, Mengenoperationen sind Logik.
- **Taktiken am Beispiel der Mengen** (3.6, 3.7): Annahmen zerlegen, Ziele zusammenbauen, umschreiben, vereinfachen, in Ketten rechnen.
- **Zahlen und Rechnen** (3.8): `ring`, `omega`, `norm_num` und Induktion.
- **Strukturen und Typklassen** (3.2): Strukturen, Monoide als Typklasse, Instanzen und `extends`.
- **Die Strukturen der Mathematik** (3.8): algebraische Hierarchie, Ordnungen, Mixins.
- Zum Schluss **Lean live** mit Blatt 3.